✂ StyleCut

Beginner's Project Guide

A step-by-step learning guide for building a full-stack salon booking web app from scratch.

Covers: React · Node.js/Express · PostgreSQL · WhatsApp Notifications

1. What You Will Build

StyleCut is a salon web application where:

Step . Customers can sign up, browse services/products, and book appointments.

Step . Barbers can log in, view their schedule, and manage bookings.

Step . WhatsApp notifications are sent to customers after booking.

■ **Tip:** Think of it as a mini Uber for haircuts — a customer-facing booking site plus a barber dashboard.

2. Technologies to Learn First

Before coding, understand what each part does:

Technology	What it does in StyleCut
React (frontend)	Builds the web pages users see — login, booking, dashboard
Node.js + Express	Backend server that handles API requests from the frontend
PostgreSQL	Database that stores bookings, users, barbers, and orders
WhatsApp Cloud API	Sends booking confirmation messages to customers via WhatsApp
npm	Package manager to install all libraries
dotenv (.env)	Stores secret keys (API tokens, DB passwords) safely

■ **Tip:** Learn HTML/CSS basics and JavaScript fundamentals before starting. Free resources: MDN Web Docs, freeCodeCamp.

3. Project Folder Structure

Your project will have three main folders:

Folder / File	Purpose
client/	All React frontend code (what users see in the browser)
client/src/App.jsx	Defines all page routes (which URL shows which page)
client/src/api.js	Functions that call backend APIs
client/src/Book.jsx	Booking page for customers
client/src/BarberDashboard.jsx	Dashboard page for barbers

server/	All backend Node.js/Express code
server/src/index.js	Main file — defines all API routes
server/src/db.js	Connects to PostgreSQL database
server/src/whatsapp.js	Sends WhatsApp messages via Meta Cloud API
server/.env	Your secret keys (never share this file!)
database/	SQL files to create your database tables

4. How Data Flows (Big Picture)

Understanding the data flow is the most important concept:

#	What happens	Which file
1	User opens the browser	client/src/App.jsx loads
2	User clicks 'Book Appointment'	Book.jsx page is shown
3	User submits the booking form	api.js sends a POST request to the backend
4	Backend receives the request	server/src/index.js route handles it
5	Backend saves booking to DB	server/src/db.js runs a SQL INSERT
6	Backend triggers notification	server/src/whatsapp.js sends WhatsApp message
7	Response sent back to frontend	Book.jsx shows 'Booking confirmed!'

■ **Tip:** Always trace a feature through all 7 steps above when debugging. Ask: where does it break?

5. Setting Up the Project

A. Install Prerequisites

Step 1. Install **Node.js** from https://nodejs.org (choose LTS version)

Step 2. Install **PostgreSQL** from https://postgresql.org/download

Step 3. Install **VS Code** (code editor) from https://code.visualstudio.com

B. Install Dependencies

Open your terminal inside the project folder and run:

```
npm install
npm install --prefix server
npm install --prefix client
```

C. Configure Environment Variables

Copy the example file and fill in your real values:

```
cp server/.env.example server/.env
```

Variable	What to put here
DATABASE_URL	Your PostgreSQL connection string
WHATSAPP_TOKEN	Your Meta WhatsApp Cloud API token
WHATSAPP_PHONE_ID	Your WhatsApp Phone Number ID from Meta Developer Portal

PORT	3000 (or any port for your backend server)

■ **Note:** Never commit your .env file to GitHub. Add it to .gitignore immediately.

D. Create the Database

Run the SQL files in the database/ folder using psql or a tool like TablePlus / pgAdmin.

E. Start the App

```
npm run dev
```

Or run frontend and backend separately:

```
npm run dev:server    # starts Express on port 3000
npm run dev:client    # starts React on port 5173
```

Then open **http://localhost:5173** in your browser.

6. Recommended Build Order for Beginners

Build the project in this order so each step builds on the last:

Phase	What to build	Skills you learn
1 — Database	Create tables: users, barbers, bookings, products	SQL, PostgreSQL
2 — Backend basics	Express server + db.js connection + test with Postman	Node.js, REST APIs
3 — Auth routes	POST /register and POST /login with password hashing	bcrypt, JWT tokens
4 — Booking route	POST /bookings — saves a booking row to DB	SQL INSERT, API design
5 — React setup	Create React app, set up routing in App.jsx	React, React Router
6 — Login pages	ClientAuth.jsx and BarberAuth.jsx forms	React forms, useState
7 — Booking page	Book.jsx — form that calls POST /bookings via api.js	fetch/axios, useEffect
8 — Dashboard	BarberDashboard.jsx — fetch and display bookings list	Real state, GET API
9 — WhatsApp	whatsapp.js — call Meta Cloud API after booking saved	Env vars, API tokens
10 — Polish	Error handling, loading states, mobile layout	UX, CSS

7. WhatsApp Notification — Quick Setup

Step 1. Go to **https://developers.facebook.com** → Create App → Select Business type

Step 2. Add **WhatsApp** product to your app

Step 3. Go to **WhatsApp** → **API Setup** — copy your **Phone Number ID** and **WABA ID**

Step 4. Generate a temporary or permanent **Access Token**

Step 5. Paste these values into your **server/.env** file

Step 6. In whatsapp.js, call the API using fetch to send a template message

■ **Tip:** Start by sending a test message to your own number using curl or Postman before writing code.

8. Key Concepts to Understand

Concept	Plain English Explanation

REST API	Backend exposes URLs (endpoints). Frontend sends GET/POST requests to them.
JWT Token	A secure string given after login. Frontend sends it with every request to prove identity.
bcrypt	A library that scrambles passwords before saving them — so even you can't read them.
CORS	A browser security rule. Your backend must allow requests from your frontend's URL.
useState	React hook to store and update data inside a component (e.g. form inputs).
useEffect	React hook to run code when a page loads (e.g. fetch bookings from the backend).
async/await	JavaScript syntax to wait for API calls to finish before continuing.
Environment vars	Secret values stored outside your code so they don't leak on GitHub.

9. Common Beginner Errors & Fixes

Error	Fix
CORS error in browser console	Add cors() middleware in server/src/index.js
Cannot connect to database	Check DATABASE_URL in .env. Is PostgreSQL running?
401 Unauthorized from API	Your JWT token is missing or expired. Re-login.
WhatsApp message not sending	Verify WHATSAPP_TOKEN and PHONE_ID in .env are correct
React page shows blank	Check browser console (F12) for errors. Check App.jsx routes.
npm install fails	Delete node_modules folder and package-lock.json, then retry

10. Learning Resources

Topic	Resource
HTML & CSS basics	https://developer.mozilla.org
JavaScript	https://javascript.info
React	https://react.dev (official docs)
Node.js + Express	https://expressjs.com + nodejs.org/docs
PostgreSQL	https://www.postgresqltutorial.com
WhatsApp Cloud API	https://developers.facebook.com/docs/whatsapp/cloud-api
Git & GitHub	https://docs.github.com/en/get-started

You now have everything you need. Start small, build one feature at a time, and test at every step. Good luck! ✂